

C Programming Coding Standard Manual

Document Reference: QA-CS-001 **Version:** 2.0 **Classification:** Internal / Quality Assurance Baseline **Applicable Domain:** Multi-Industry High-Integrity & Safety-Critical C Software (Automotive, Defence, Aerospace, Rail, Industrial)

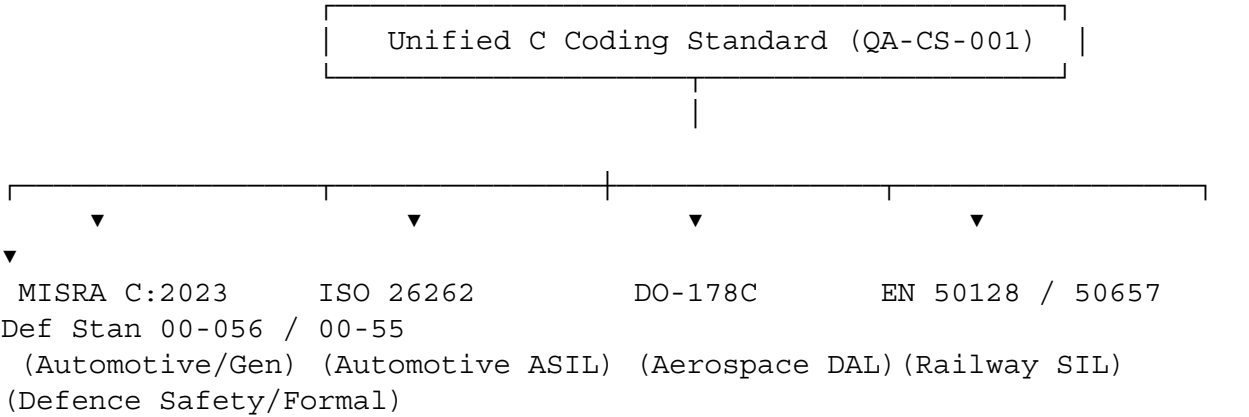
1. Document Control & Metadata

1.1 Revision History

Version	Date	Author	Description
1.0	DD-MM-YYYY	QA Team / Software Lead	Initial Baseline (MISRA C:2023 / Automotive focus)
2.0	24-07-2026	QA Systems Engineering	Multi-Domain Harmonization (DO-178C, Def Stan 00-056/00-55, EN 50128/50657, IEC 61508)

1.2 Standard Cross-Reference Matrix

This coding standard harmonizes requirements across international domain-specific safety, security, and quality standards:



Standard / Standard Body	Applicable Domain	Relevance & Application
MISRA C:2023	General / Automotive	Core rule baseline, language subsets, concurrency guidelines
ISO 26262 (ASIL A-D)	Automotive	Functional Safety, SWE.3/SWE.4 static analysis

Standard / Standard Body	Applicable Domain	Relevance & Application
		compliance
RTCA DO-178C / ED-12C (DAL A-E)	Aerospace & Defence Airborne	Software verification, low-level testing, determinism, traceability
EN 50128 / EN 50657 (SIL 0-4)	Railway Applications	Signal/Rolling stock protection, formal static analysis, structural testing
UK Def Stan 00-056 / 00-55	Defence Equipment	Hazard analysis, formal specification, target safety cases
CERT C / ISO/IEC 9899	Cyber Security / C Standard	Secure coding standards (C90/C99/C11/C18 language rules)

2. Purpose & Scope

2.1 Purpose

This standard mandates safe, secure, reliable, deterministic, and testable C programming practices. Objectives include:

- Eliminating Undefined, Unspecified, and Implementation-Defined behavior.
- Guaranteeing predictable execution timing and stack boundaries.
- Streamlining verification across independent regulatory audits (FAA, EASA, CENELEC, MOD).

2.2 Scope

Applicable to all embedded C codebases across target domains:

- **Target Systems:** Application Software, Bootloaders, BSPs, Device Drivers, Middleware, RTOS Interfaces, Diagnostic Stacks, AUTOSAR/POSIX Modules, and Safety-Critical Core Logic.
- **Exclusions:** Third-party binary drivers or pre-qualified legacy code require formal safety-case justification and sandbox wrapper isolates.

3. Language Subsetting & Compiler Settings

3.1 Supported C Standards

- ISO/IEC 9899:1990 (C90), ISO/IEC 9899:1999 (C99), ISO/IEC 9899:2011 (C11), ISO/IEC 9899:2018 (C18).
- Language extensions (e.g., `__attribute__`, inline assembly) are **strictly prohibited** unless explicitly isolated inside documented abstraction wrappers and approved via a formal deviation process.

3.2 Compiler Configuration

- **Warning Levels:** Enable maximum diagnostics (-Wall -Wextra -Wpedantic -Wshadow -Wconversion -Wstrict-prototypes).
- **Treat Warnings as Errors:** All warnings shall be treated as errors (-Werror). Zero warnings allowed in release builds.
- **Optimization:** Optimizations affecting evaluation order or stack layout must be validated on actual hardware. Stack checking mechanisms (-fstack-protector-all) shall be activated.

4. Source Formatting & Layout

4.1 Indentation & Spacing

- **Indentation:** Exactly **4 spaces** per level. Hard tabs are prohibited (\t).
- **Line Width:** Standard limit is **120 characters** (maximum 80 characters for complex multi-operand conditional statements to aid readability side-by-side).
- **Braces:** Braces are mandatory for all control constructs (if, else, while, for, do-while, switch). Opening brace must sit on a new line aligned with the control statement.

```
/* Compliant Brace Layout */
if (isActive == true)
{
    ExecuteTask();
}
else
{
    /* Fallback logic */
}
```

4.2 File Header & Standard Structure

Every source (.c) and header (.h) file must begin with a standardized header block:

```
/* *****
*****
* Company Name      : <Company / Organization Name>
* File Name         : system_monitor.c
* Module            : Health Monitoring Unit
* Author            : QA / Embedded Systems Engineering
* Standard Ref      : MISRA C:2023 / DO-178C DAL-A / EN 50128 SIL-4
* Description       : High-integrity system status verification module.
* Copyright         : Copyright (c) 2026 <Company Name>. All Rights
Reserved.
* Target Domain     : Aerospace / Defence / Rail / Automotive
*****
*****/
```

5. Naming Conventions

All identifiers must follow clear, unambiguous naming rules to eliminate misinterpretations.

Target Identifier	Prefix / Casing Format	Example	Description / Notes
Global Variable	g_ + camelCase	g_vehicleSpeed	Minimize global usage; require access locks
Static Variable	s_ + camelCase	s_retryCounter	Module-scoped variable
Pointer Variable	p + CamelCase	pBuffer	Must be explicitly checked against NULL
Constants / Macros	UPPER_CASE_SNAKE	MAX_BUFFER_SIZE	Macro/Constant definitions
Type Definitions	PascalCase_t	SensorStatus_t	Typedef structures, unions, enums
Functions	Module_PascalCase()	Sensor_ReadVal()	Explicit, action-oriented verb name
Enum Constants	PREFIX_UPPER_CASE	STATE_RUNNING	Domain-prefixed enum members

6. Type Definitions, Variables, and Constants

6.1 Fixed-Width Integer Types

Standard C basic types (int, short, long, char) are **prohibited** for numerical data due to architecture dependencies. Developers shall explicitly import <stdint.h> and <stdbool.h>.

```
/* Compliant Types */
uint8_t    u8Index = 0U;
uint16_t   u16Speed = 0U;
uint32_t   u32Timeout = 0U;
int32_t     s32ErrorCode = 0;
bool       bIsValid = false;

/* Non-Compliant Types */
int speed;      /* NON-COMPLIANT: Implicit size */
long duration; /* NON-COMPLIANT: Platform-dependent */
```

6.2 Variable Initialization & Scope

- **Initialization:** Every variable shall be initialized at its point of declaration. Uninitialized variables are strictly forbidden.
- **Scope & Lifetime:** Declare variables in the innermost block possible.

6.3 Constants & Literals

- Magic numbers are strictly prohibited. Use #define or static const expressions.
- Unsigned integer constants must carry a U suffix.

```
/* Non-Compliant */
if (speed > 45) { /* Magic Number */ }
```

```

/* Compliant */
#define SPEED_LIMIT_KMPH (45U)
if (speed > SPEED_LIMIT_KMPH) { ... }

```

7. Control Flow & Defensive Rules

7.1 Single Exit Logic & Branching

- Functions should ideally have a single exit point (single return statement at the end of the function) to prevent unexecuted cleanup code.
- Every if ... else if sequence **shall** end with a terminal else block containing explicit fallback logic or an explanatory comment.

```

/* Compliant If-Else Structure */
if (systemMode == MODE_INIT)
{
    InitSystem();
}
else if (systemMode == MODE_RUN)
{
    ExecuteSystem();
}
else
{
    /* Safe State Recovery */
    TriggerSafeState();
}

```

7.2 Switch Statements

- Every switch statement **must** feature an explicit default: label.
- Every non-empty case block must terminate with an explicit break; statement. Intentional fall-through must be annotated with standard directives.

7.3 Unconstrained Jumps & Iteration Control

- The goto construct is **strictly prohibited**.
- break and continue constructs shall not be used to prematurely break execution out of standard iteration loops (for, while) unless formally justified. Loop execution flags must govern loop exit.

8. Functions, Pointers, Arrays, and Dynamic Memory

8.1 Functions

- **Complexity:** Cyclomatic Complexity (CC) shall be **≤ 10** (Hard ceiling <20 under strict static checks).
- **Function Length:** Maximum recommended length is **50–100 lines**.
- **Parameters:** Maximum **4 parameters** passed to any single function. Pass structures via pointer-to-const if larger parameters are required.
- **Recursion:** Recursion (direct or indirect) is **strictly prohibited**.

8.2 Pointer Rules & Memory Management

- **Null Checks:** Pointers **must** be validated against NULL prior to dereferencing.
- **Pointer Arithmetic:** Explicit pointer arithmetic (e.g., *(ptr + i)) is forbidden. Use formal array indexing (ptr[i]) after verifying array bounds.
- **Dynamic Memory:** Dynamic memory allocations (`malloc`, `calloc`, `realloc`, `free`) are **strictly prohibited** in runtime safety software. Use static buffers allocated at compile time.

```
/* Compliant Buffer Access */
void ProcessData(const uint8_t * const pInputBuffer, uint16_t length)
{
    if ((pInputBuffer != NULL) && (length <= MAX_BUFFER_SIZE))
    {
        for (uint16_t i = 0U; i < length; i++)
        {
            ProcessElement(pInputBuffer[i]);
        }
    }
}
```

9. Domain-Specific Extensions

9.1 Defence & Aerospace Extensions (DO-178C DAL-A/B & Def Stan 00-056 / 00-55)

- **Formal Verification:** In accordance with Def Stan 00-55 / 00-56 (SIL 3/4), key state machines and safety mechanisms shall undergo formal methods analysis and Design-by-Contract contract checks.
- **Traceability Matrix:** Every low-level software function **must link directly** to a System High-Level Requirement (HLR) or Low-Level Requirement (LLR) in DO-178C DOORS/Polarion tools. Unmapped functions are treated as dead code.
- **Bounded Execution Time:** Functions executing in real-time interrupt contexts must have strict Worst-Case Execution Time (WCET) bounds.

9.2 Railway Safety Extensions (EN 50128 / EN 50657 SW-SIL 3/4)

- **No Unintended Side Effects:** Expressions shall not combine assignments and logical checks.
- **Defensive Checks:** Data received from external communication busses (e.g., MVB, CAN, Ethernet) must undergo continuous sanity checks and frame integrity verification

before execution.

10. QA Verification, Static Analysis, & Deviation Management

10.1 Required Static Analysis Engine Stack

Projects must integrate qualified static analysis engines into the automated CI/CD pipeline:

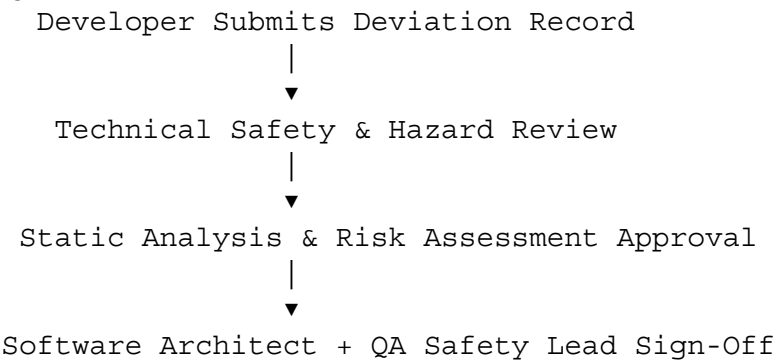
- **Commercial Engines:** Helix QAC, Polyspace Bug Finder/Prover, Coverity, LDRA Testbed, PC-lint Plus.
- **Open / Supplementary:** Cppcheck.

10.2 Software Quality Targets

Metric	Threshold / Target	Action upon Breach
Compiler Warnings	Exactly 0 Warnings	Build Failure
MISRA Mandatory Violations	Exactly 0 Violations	Build Failure
MISRA Required Violations	0 (or Formal Approved Deviation)	Review Escalation
Cyclomatic Complexity	≤ 10 per function	Refactoring Required
Code Coverage (DAL-A / SIL-4)	100% MC/DC (Modified Condition/Decision)	Block Release
Code Coverage (ASIL-B / DAL-C)	100% Statement + Branch Coverage	Block Release

10.3 Formal Deviation Workflow

If a MISRA or domain rule must be breached (e.g., mapping absolute addresses to hardware registers):



Deviation Template Example

- **Deviation ID:** DEV-MISRA-2026-001
- **Rule Violated:** MISRA C:2023 Rule 11.4 / Rule 11.6 (Pointer Cast to Integer)
- **Location:** drivers/timer_hw.c (Line 42)
- **Technical Justification:** Required to directly access hardware memory-mapped register

address 0x40000000.

- **Safety & Hazard Mitigation:** Register pointer volatile-qualified and constrained in driver wrapper module. Verified via hardware unit tests.
- **Approval Authority:** Software Architect & QA Lead Sign-Off.

11. Quality Assurance Code Review Checklist

Verification Item	Requirement Description	Compliance Status
Compiler Build	Compiles with -Werror without warnings.	☐ Pass
MISRA Compliance	Zero Mandatory violations; Required violations formally deviated.	☐ Pass
Header Integrity	Includes standard company copyright, license, and change log.	☐ Pass
Type Safety	Pure <stdint.h> implementation; no raw scalar types.	☐ Pass
Variable Safety	All variables explicitly initialized; scope minimized.	☐ Pass
Pointer Validation	Checked against NULL prior to dereferencing; zero dynamic memory.	☐ Pass
Defensive Logic	Terminal else present; switch has explicit default.	☐ Pass
Traceability	Functions map to system software requirements (DO-178C/EN 50128).	☐ Pass